Os Princípios S.O.L.I.D em Imagens

Por **Ugonna Thelma** · Backticks & Tildes

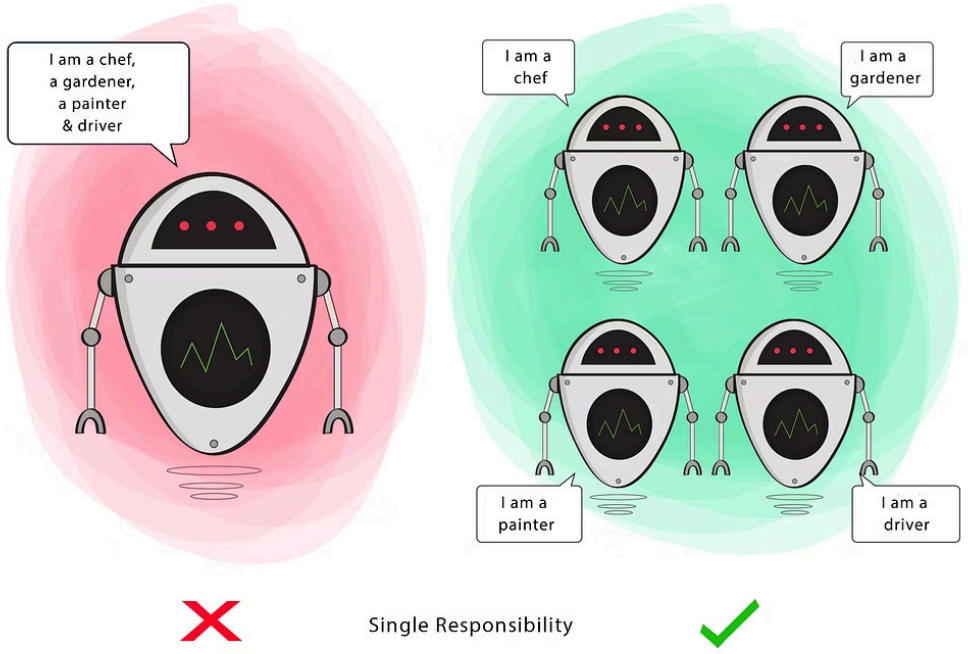
Os cinco princípios de desenvolvimento de software **SOLID** são diretrizes para construir software mais fácil de escalar e manter. Popularizados pelo engenheiro **Robert C. Martin**, eles ajudam a tornar o código mais fácil de ajustar, estender e testar.

A sigla....

1. S — Responsabilidade Única
2. O — Aberto-Fechado
3. L — Substituição de Liskov
4. I — Segregação de Interface
5. D — Inversão de Dependência

S — RESPONSABILIDADE ÚNICA

Uma classe deve ter uma única responsabilidade



O Problema

Se uma classe tem muitas responsabilidades, aumenta a possibilidade de bugs. Alterar uma responsabilidade pode afetar as outras sem que você perceba.

O Objetivo

Separar comportamentos para que, se bugs surgirem por causa de uma mudança, eles não afetem outros comportamentos não relacionados.

O — ABERTO-FECHADO

Classes devem ser abertas para extensão, mas fechadas para modificação

O Problema

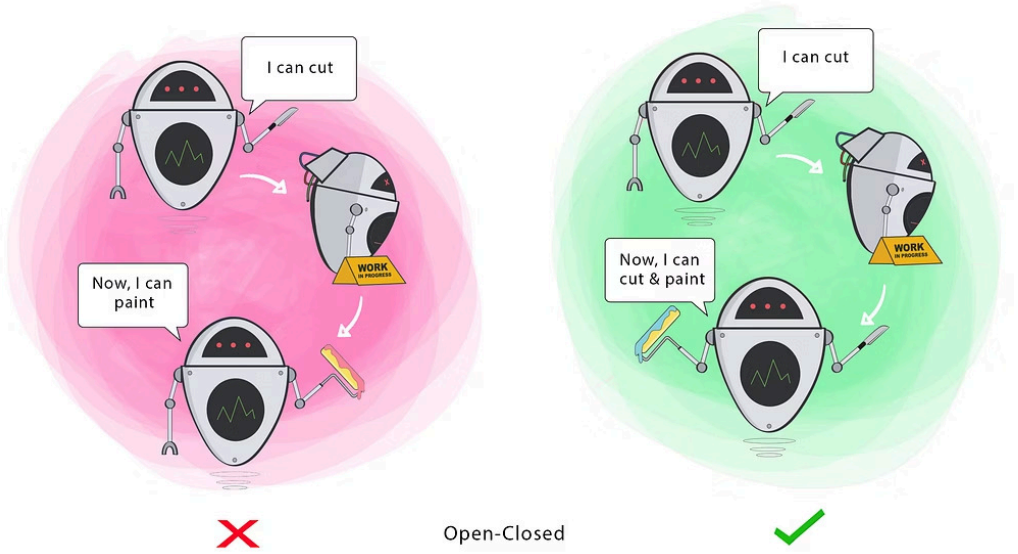
Alterar o comportamento atual de uma classe afetará todos os sistemas que a utilizam.

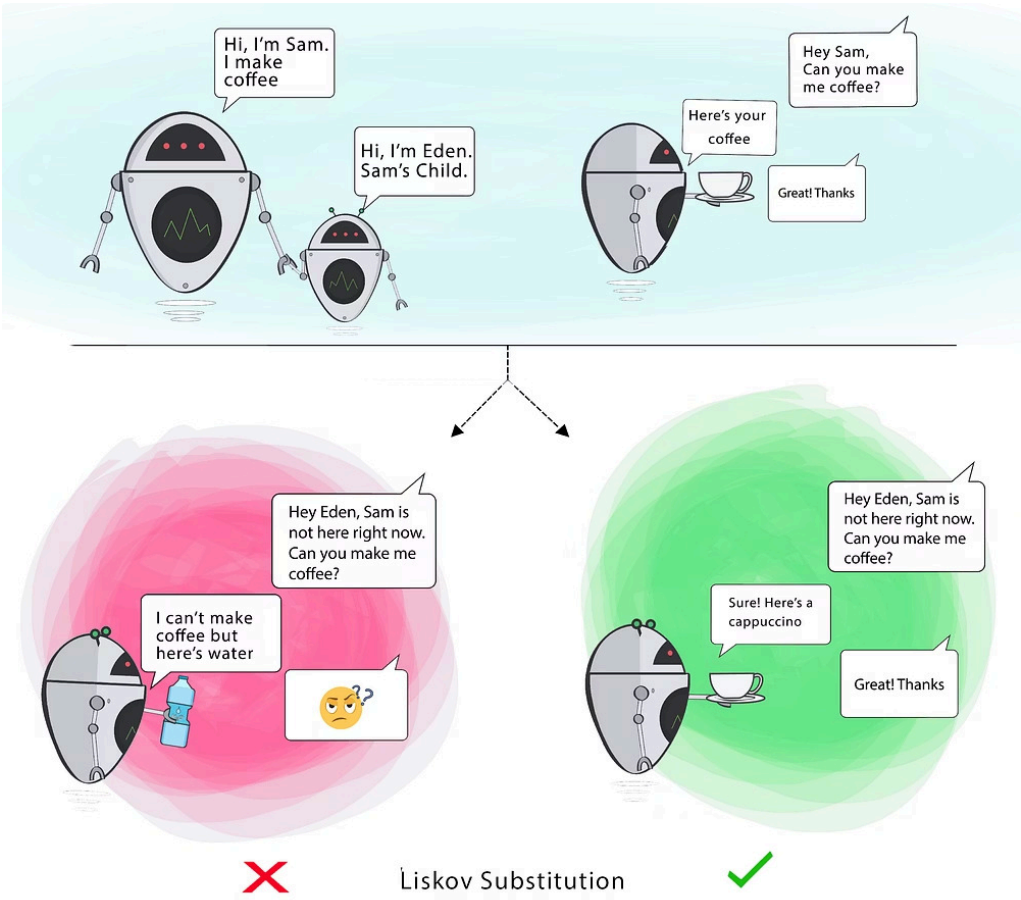
A Solução

Se você quer que a classe execute mais funções, o ideal é **adicionar** às funções existentes, e não modificá-las.

O Objetivo

Estender o comportamento de uma classe sem alterar o comportamento existente, evitando bugs onde a classe é utilizada.





L — SUBSTITUIÇÃO DE LISKOV

Objetos de um subtipo devem poder substituir objetos do tipo pai

Como funciona

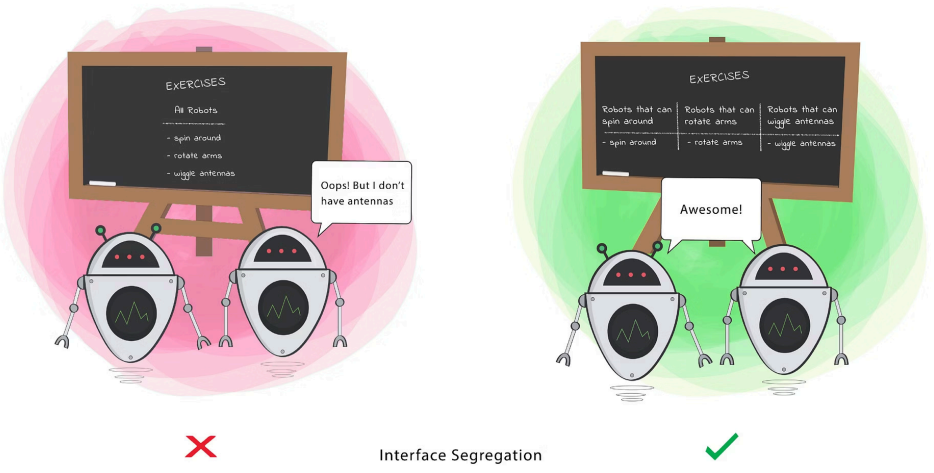
Quando uma classe **filha** não consegue executar as mesmas ações que a classe **pai**, isso pode causar bugs. A classe filha deve processar os mesmos pedidos e entregar o mesmo tipo de resultado.

O Objetivo

Garantir consistência para que a classe pai ou filha possam ser usadas da mesma forma sem erros. Na ilustração: entregar Cappuccino (tipo de café) é aceitável; entregar Água, não.

I — SEGREGAÇÃO DE INTERFACE

Clientes não devem ser forçados a depender de métodos que não utilizam



O Problema

Quando uma classe é obrigada a executar ações que não são úteis para ela, isso é um desperdício e pode produzir bugs inesperados.

O Objetivo

Dividir um conjunto de ações em conjuntos menores para que uma classe execute **apenas** o conjunto de ações que ela realmente necessita. Qualquer outra ação deve ser removida ou movida para outro lugar.

D — INVERSÃO DE DEPENDÊNCIA

Módulos de alto nível não devem depender de módulos de baixo nível

Módulo de Alto Nível

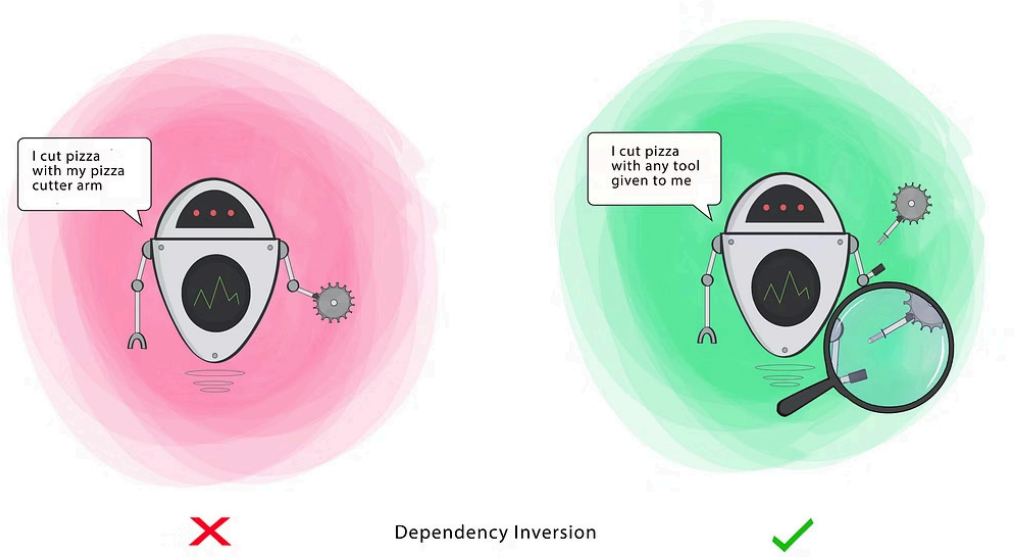
A classe que executa uma ação com uma ferramenta.

Abstração

A interface que conecta as duas classes.

Objetivo

Reduzir a dependência da classe de alto nível na de baixo nível introduzindo uma interface.



Resumo dos Princípios SOLID

Esses cinco princípios ajudam a tornar o código mais fácil de ajustar, estender e testar com poucos ou nenhum problema.

1	**Responsabilidade Única** Separe comportamentos para isolar o impacto de mudanças.
2	**Aberto-Fechado** Estenda funcionalidades sem modificar o comportamento existente.
3	**Substituição de Liskov** Classes filhas devem poder substituir as classes pai sem erros.
4	**Segregação de Interface** Classes devem executar apenas as ações que realmente necessitam.
5	**Inversão de Dependência** Dependa de abstrações, não de implementações concretas.

Ilustrações originais por **Ugonna Thelma**. Para uso profissional ou impressão, acesse: **https://medium.com/backticks-tildes/the-s-o-l-i-d-principles-in-pictures-b34ce2f1e898**